Assessment Test

HCLID- HCLTFP2283133

Name - Madde Pavan

USN - 23BTLEI001

Electronics and communication engineering

SECTION A: ADVANCED SQL

Question 1: Database Design and Advanced Queries (8 Marks)

a. Display employee names, department names, and manager names using joins. (2 marks)

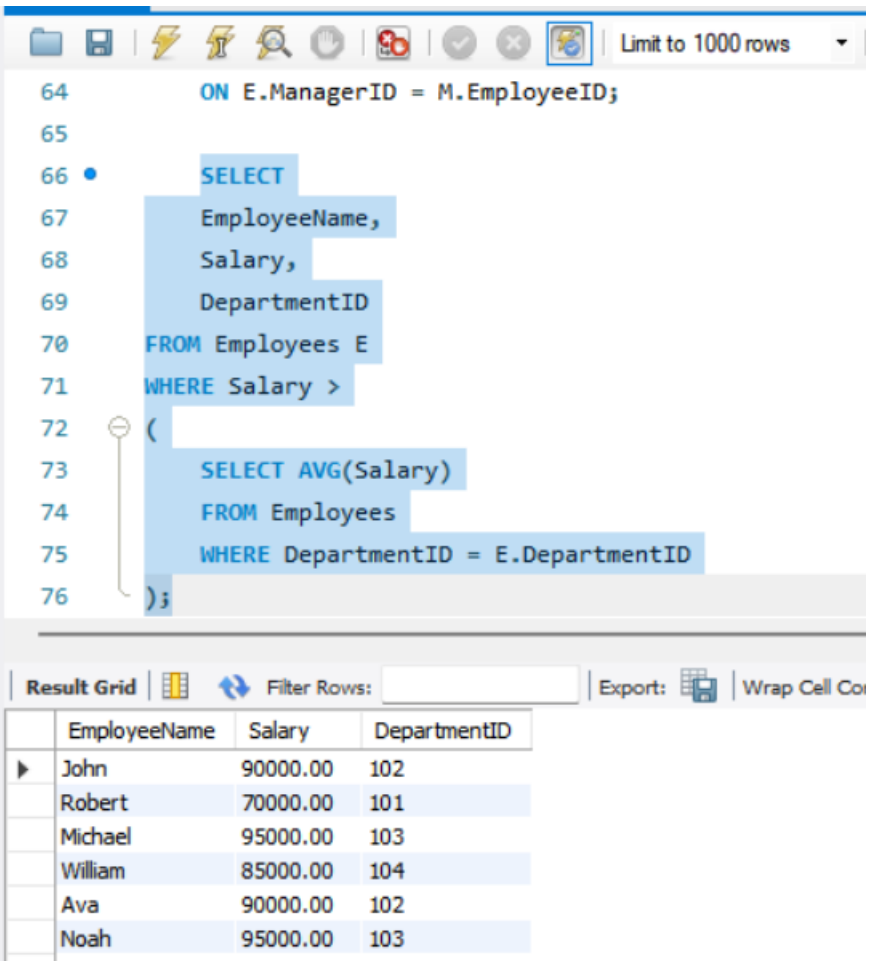
```
56 •  SELECT
57        E.EmployeeName AS Employee,
58        D.DepartmentName,
59        M.EmployeeName AS Manager
60    FROM Employees E
61    JOIN Departments D
62        ON E.DepartmentID = D.DepartmentID
63    LEFT JOIN Employees M
64        ON E.ManagerID = M.EmployeeID;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

Employee	DepartmentName	Manager
Robert	HR	NULL
Sophia	HR	Robert
Daniel	HR	Robert
John	IT	NULL
David	IT	John
Emily	IT	John
Ava	IT	John

Result 2

b. Find employees earning more than their department's average salary. (2 marks)



```
64          ON E.ManagerID = M.EmployeeID;
65
66          SELECT
67          EmployeeName,
68          Salary,
69          DepartmentID
70      FROM Employees E
71      WHERE Salary >
72      (
73          SELECT AVG(Salary)
74          FROM Employees
75          WHERE DepartmentID = E.DepartmentID
76      );
```

Result Grid | Filter Rows: | Export: | Wrap Cell Co

EmployeeName	Salary	DepartmentID
John	90000.00	102
Robert	70000.00	101
Michael	95000.00	103
William	85000.00	104
Ava	90000.00	102
Noah	95000.00	103

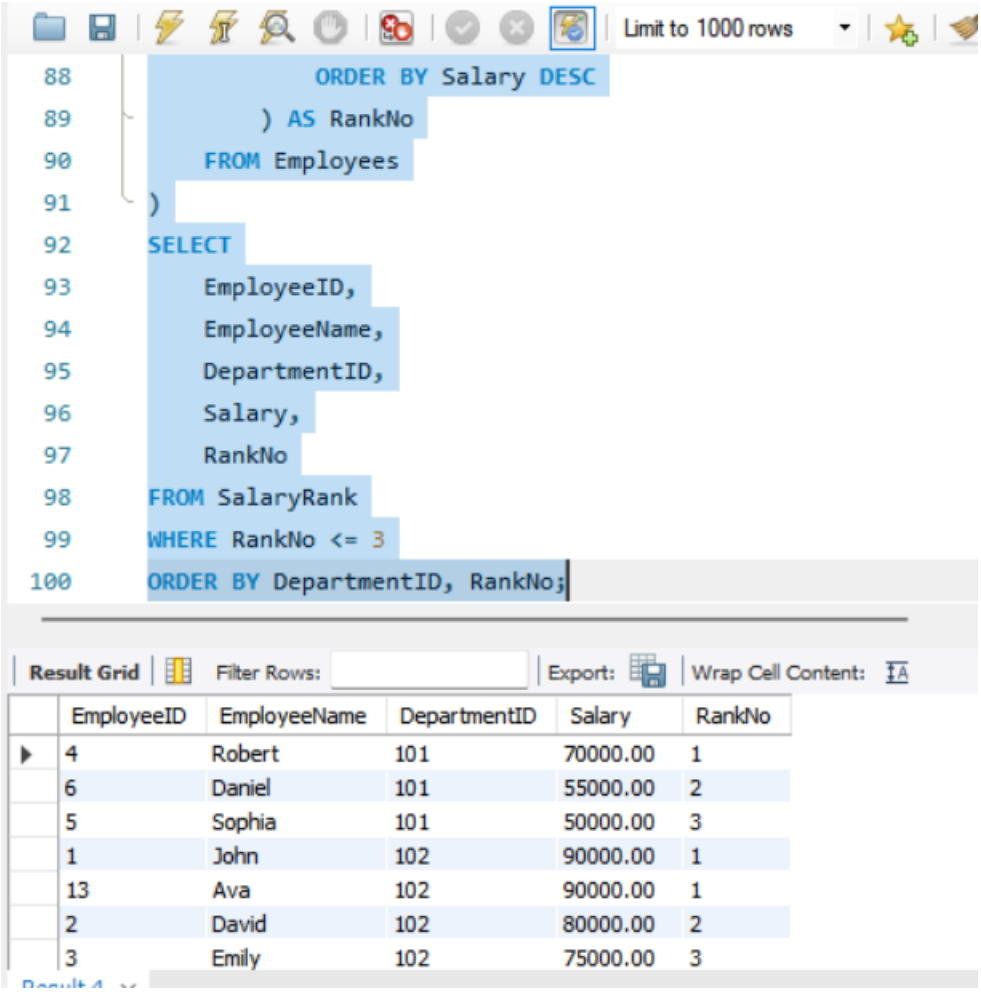
c. Find the second-highest distinct salary without using LIMIT, TOP, or FETCH FIRST. (2 marks)

```
101
102 •  SELECT MAX(Salary) AS SecondHighestSalary
103    FROM Employees
104    WHERE Salary <
105    (
106        SELECT MAX(Salary)
107        FROM Employees
108    );
```

Result Grid | Filter Rows: | Export: | Wrap

SecondHighestSalary
90000.00

d.Rank employees by salary within each department and display the top three employees per department, retaining ties at the cutoff. (2 marks)



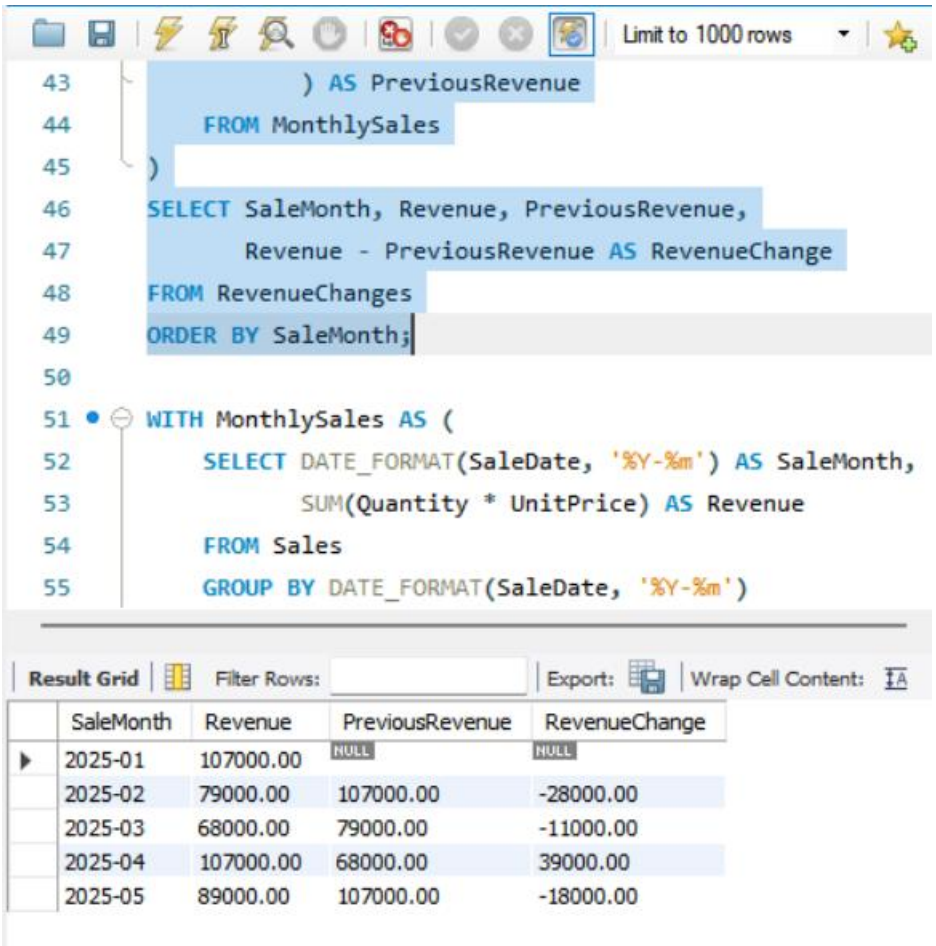
```
88              ORDER BY Salary DESC
89          ) AS RankNo
90      FROM Employees
91  )
92  SELECT
93      EmployeeID,
94      EmployeeName,
95      DepartmentID,
96      Salary,
97      RankNo
98  FROM SalaryRank
99  WHERE RankNo <= 3
100 ORDER BY DepartmentID, RankNo;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

EmployeeID	EmployeeName	DepartmentID	Salary	RankNo
4	Robert	101	70000.00	1
6	Daniel	101	55000.00	2
5	Sophia	101	50000.00	3
1	John	102	90000.00	1
13	Ava	102	90000.00	1
2	David	102	80000.00	2
3	Emily	102	75000.00	3

Question 2: Sales Analytics and Query Optimization (6 Marks)

a. Calculate total revenue for each product. (2 marks)



```
43              ) AS PreviousRevenue
44          FROM MonthlySales
45      )
46      SELECT SaleMonth, Revenue, PreviousRevenue,
47             Revenue - PreviousRevenue AS RevenueChange
48      FROM RevenueChanges
49      ORDER BY SaleMonth;
50
51  •   WITH MonthlySales AS (
52          SELECT DATE_FORMAT(SaleDate, '%Y-%m') AS SaleMonth,
53                 SUM(Quantity * UnitPrice) AS Revenue
54          FROM Sales
55          GROUP BY DATE_FORMAT(SaleDate, '%Y-%m')
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

SaleMonth	Revenue	PreviousRevenue	RevenueChange
2025-01	107000.00	NULL	NULL
2025-02	79000.00	107000.00	-28000.00
2025-03	68000.00	79000.00	-11000.00
2025-04	107000.00	68000.00	39000.00
2025-05	89000.00	107000.00	-18000.00

b. Calculate monthly revenue and month-over-month revenue change using a window function. (2 marks)

Query 1 | SQL File 1*

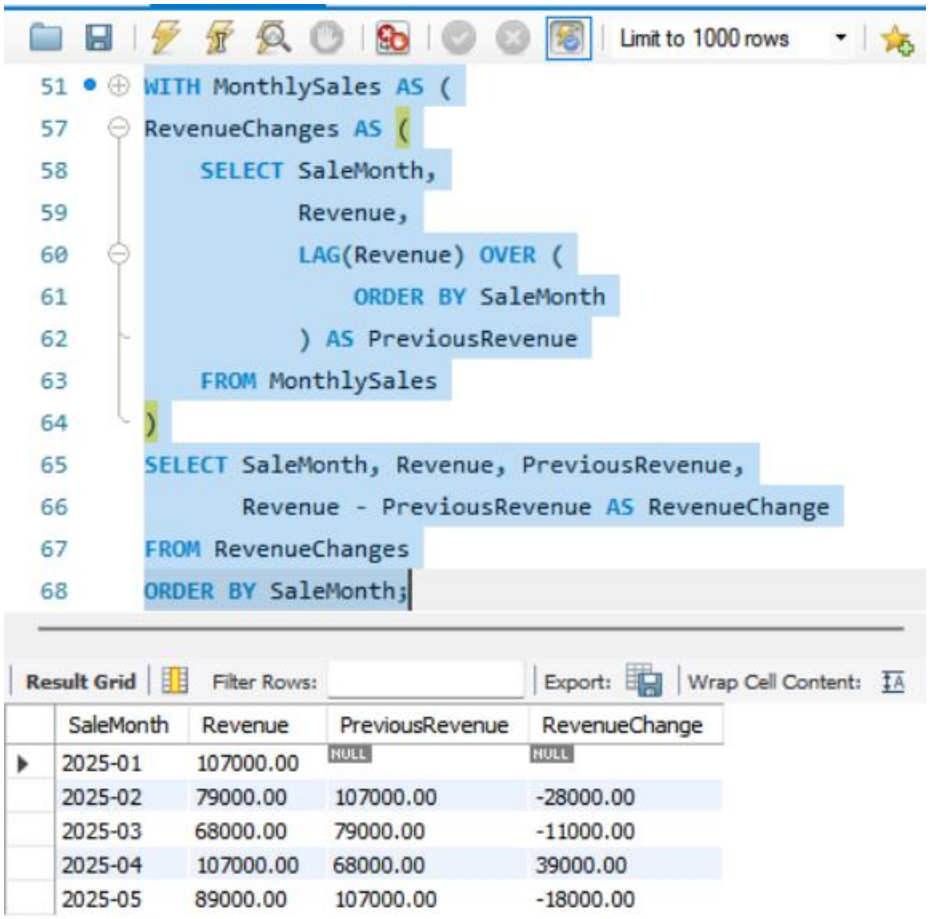
Limit to 1000 rows

```
43              ) AS PreviousRevenue
44          FROM MonthlySales
45      )
46      SELECT SaleMonth, Revenue, PreviousRevenue,
47             Revenue - PreviousRevenue AS RevenueChange
48      FROM RevenueChanges
49      ORDER BY SaleMonth;
50
51  •   WITH MonthlySales AS (
57      RevenueChanges AS (
58          SELECT SaleMonth,
59                 Revenue,
60                 LAG(Revenue) OVER (
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

SaleMonth	Revenue	PreviousRevenue	RevenueChange
2025-01	107000.00	NULL	NULL
2025-02	79000.00	107000.00	-28000.00
2025-03	68000.00	79000.00	-11000.00
2025-04	107000.00	68000.00	39000.00
2025-05	89000.00	107000.00	-18000.00

c.Rewrite the following query to improve the potential for index usage and explain why your version may perform better. (2 marks)



SaleMonth	Revenue	PreviousRevenue	RevenueChange
2025-01	107000.00	NULL	NULL
2025-02	79000.00	107000.00	-28000.00
2025-03	68000.00	79000.00	-11000.00
2025-04	107000.00	68000.00	39000.00
2025-05	89000.00	107000.00	-18000.00

Question 3: SQL Troubleshooting (6 Marks)

a. **Explain how an execution plan helps identify expensive operations in a query. (2 marks)**

An execution plan shows how the database executes a query. It helps identify:

- Full table scans and inefficient index usage.
- Expensive joins and sorting operations.
- High-cost operations and estimated row counts.
- Opportunities to add indexes or rewrite queries.

For example, `EXPLAIN SELECT * FROM Employees WHERE DepartmentID = 1;` displays the planned access method in many SQL databases.

b. **Recommend an appropriate index strategy for queries filtering employees by DepartmentID and Salary. Explain your reasoning. (2 marks)**

- Full table scans and inefficient index usage.
- Expensive joins and sorting operations.
- High-cost operations and estimated row counts.
- Opportunities to add indexes or rewrite queries.

For example, `EXPLAIN SELECT * FROM Employees WHERE DepartmentID = 1;` displays the planned access method in many SQL databases.

Write a query to identify duplicate customer email addresses and display only addresses occurring more than once. (2 marks)

Assume a table named `Customers` with an `Email` column.

sql

```
SELECT Email, COUNT(*) AS Occurrences
FROM Customers
WHERE Email IS NOT NULL
GROUP BY Email
HAVING COUNT(*) > 1;
```

SECTION B: LINUX || Maximum Marks:

15 Question 4: Log Analysis (5 Marks) Assume /var/log/application.log contains millions of log entries in the following format: 2026-10-28 10:15:23 ERROR payment-service Transaction failed Write Linux commands to:

a. Find all lines containing ERROR. (1 mark)

```
grep "ERROR" /var/log/application.log
```

b. Display the 20 most recent error entries in the file.

```
grep "ERROR" /var/log/application.log | tail -n 20
```

c. **Count the total number of error entries. (1 mark)**

```
grep -c "ERROR" /var/log/application.log
```

d. **Extract error entries for a specified date into a separate file. (1 mark)**

```
grep "^2026-10-28 .*ERROR" /var/log/application.log \
> errors_2026-10-28.log
```

e. Explain how you would calculate the number of errors occurring in each hour. (1 mark)

grep "ERROR" /var/log/application.log |

awk '{print substr($2,1,2)}' |

sort | uniq -c

Question 5: Permissions and System Troubleshooting (5 Marks)

a. Create a group named finance and a report file, assuming you have the required privileges. (1 mark)
sudo groupadd finance
touch report.txt

b. Set permissions so the owner can read and write, the finance group can read, and other users have no access. (1 mark)
sudo chgrp finance report.txt

c. Identify the processes consuming the most CPU. (1 mark)

top

ps aux --sort=-%cpu | head

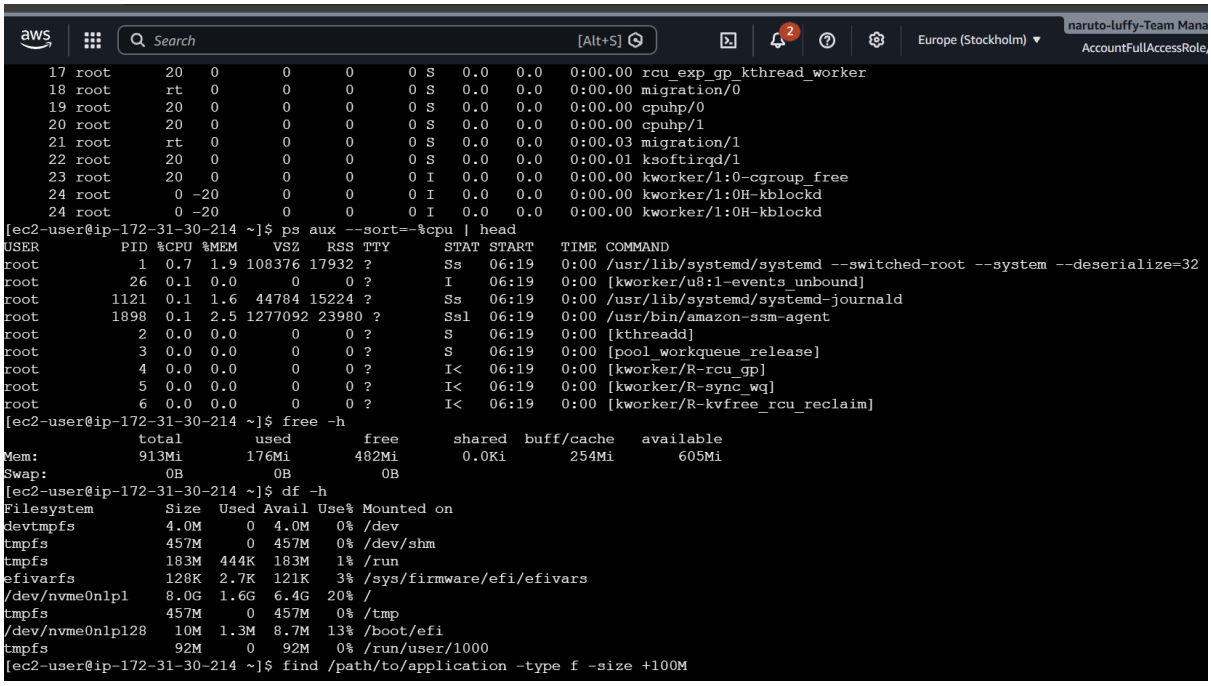
d Display memory usage and filesystem disk utilization. (1 mark)

free -h

df -h

e Find files larger than 100 MB under a specified application directory. (1 mark)

find /path/to/application -type f -size +100M



Question 7: PySpark DataFrame Operations (6 Marks)

Question 7: PySpark DataFrame Operations (6 Marks)

Create a PySpark DataFrame containing the following columns:

a. Display the schema and first five records. (1 mark)

```
First Five Records:
+----------+------------+----------+------+-----------+
|EmployeeID|EmployeeName|Department|Salary|JoiningDate|
+----------+------------+----------+------+-----------+
|         1|        John|        IT| 90000| 2018-01-15|
|         2|       David|        IT| 80000| 2019-03-10|
|         3|       Emily|        IT| 75000| 2020-06-20|
|         4|      Robert|        HR| 70000| 2017-08-12|
|         5|      Sophia|        HR| 50000| 2021-02-25|
+----------+------------+----------+------+-----------+
only showing top 5 rows
```

b. Filter employees whose monthly salary is greater than ₹60,000. (1 mark)

```
Employees with Salary > 60000:
+----------+------------+----------+------+-----------+
|EmployeeID|EmployeeName|Department|Salary|JoiningDate|
+----------+------------+----------+------+-----------+
|         1|        John|        IT| 90000| 2018-01-15|
|         2|       David|        IT| 80000| 2019-03-10|
|         3|       Emily|        IT| 75000| 2020-06-20|
|         4|      Robert|        HR| 70000| 2017-08-12|
|         7|     Michael|   Finance| 95000| 2016-04-18|
|         8|      Olivia|   Finance| 65000| 2020-09-11|
|         9|       James|   Finance| 70000| 2021-11-15|
|        10|     William|     Sales| 85000| 2018-07-09|
|        12|        Liam|     Sales| 62000| 2023-01-20|
|        13|         Ava|        IT| 90000| 2023-02-10|
|        14|        Noah|   Finance| 95000| 2022-03-05|
|        15|         Mia|     Sales| 85000| 2024-01-05|
```

c. Add an AnnualSalary column calculated from the monthly salary. (1 mark)

```
DataFrame with AnnualSalary:
+----------+------------+----------+------+-----------+------------+
|EmployeeID|EmployeeName|Department|Salary|JoiningDate|AnnualSalary|
+----------+------------+----------+------+-----------+------------+
|         1|        John|        IT| 90000| 2018-01-15|     1080000|
|         2|       David|        IT| 80000| 2019-03-10|      960000|
|         3|       Emily|        IT| 75000| 2020-06-20|      900000|
|         4|      Robert|        HR| 70000| 2017-08-12|      840000|
|         5|      Sophia|        HR| 50000| 2021-02-25|      600000|
|         6|      Daniel|        HR| 55000| 2022-01-14|      660000|
|         7|     Michael|   Finance| 95000| 2016-04-18|     1140000|
|         8|      Olivia|   Finance| 65000| 2020-09-11|      780000|
|         9|       James|   Finance| 70000| 2021-11-15|      840000|
|        10|     William|     Sales| 85000| 2018-07-09|     1020000|
|        11|        Emma|     Sales| 60000| 2022-05-16|      720000|
|        12|        Liam|     Sales| 62000| 2023-01-20|      744000|
|        13|         Ava|        IT| 90000| 2023-02-10|     1080000|
|        14|        Noah|   Finance| 95000| 2022-03-05|     1140000|
|        15|         Mia|     Sales| 85000| 2024-01-05|     1020000|
+----------+------------+----------+------+-----------+------------+
```

d.Group employees by department and calculate the average salary and employee count. (2 marks)

```
Department-wise Average Salary and Employee Count:
+----------+------------------+-------------+
|Department|     AverageSalary|EmployeeCount|
+----------+------------------+-------------+
|        IT|           83750.0|            4|
|        HR|58333.333333333336|            3|
|   Finance|           81250.0|            4|
|     Sales|           73000.0|            4|
+----------+------------------+-------------+
```

e.Display the five highest-paid employees. (1 mark)

```
Top 5 Highest Paid Employees:
+----------+------------+----------+------+
|EmployeeID|EmployeeName|Department|Salary|
+----------+------------+----------+------+
|        14|        Noah|   Finance| 95000|
|         7|     Michael|   Finance| 95000|
|        13|         Ava|        IT| 90000|
|         1|        John|        IT| 90000|
|        10|     William|     Sales| 85000|
+----------+------------+----------+------+
```

Question 9: PySpark Joins and Performance (7 Marks)

a. Load both datasets into DataFrames. (1 mark)

```
df = spark.read.csv("transactions.csv", header=True, schema=schema)
df.show()
```

```
+-------------+----------+---------------+-------+--------+---------+
|TransactionID|CustomerID|TransactionDate|Product|Quantity|UnitPrice|
+-------------+----------+---------------+-------+--------+---------+
|            1|     101.0|     2024-01-10| Laptop|    NULL|  50000.0|
|            2|     102.0|     2024-01-12| Mobile|    NULL|  20000.0|
|            3|     103.0|     15/01/2024| Tablet|    NULL|  15000.0|
|            2|     102.0|     2024-01-12| Mobile|    NULL|  20000.0|
|            4|     104.0|        invalid| Laptop|    NULL|  50000.0|
|            5|      NULL|     2024-02-01| Tablet|    NULL|  15000.0|
|            6|     106.0|     2024-02-05|   NULL|    NULL|  10000.0|
|            7|     107.0|     2024-02-10| Mobile|    NULL|  20000.0|
+-------------+----------+---------------+-------+--------+---------+
```

Join the datasets to display orders with customer details. (2 marks)

```
df_cleaned = df.dropDuplicates(["TransactionID"])
df_cleaned.show()
```

```
+-------------+----------+---------------+-------+--------+---------+
|TransactionID|CustomerID|TransactionDate|Product|Quantity|UnitPrice|
+-------------+----------+---------------+-------+--------+---------+
|            1|     101.0|     2024-01-10| Laptop|    NULL|  50000.0|
|            2|     102.0|     2024-01-12| Mobile|    NULL|  20000.0|
|            3|     103.0|     15/01/2024| Tablet|    NULL|  15000.0|
|            4|     104.0|        invalid| Laptop|    NULL|  50000.0|
|            5|      NULL|     2024-02-01| Tablet|    NULL|  15000.0|
|            6|     106.0|     2024-02-05|   NULL|    NULL|  10000.0|
|            7|     107.0|     2024-02-10| Mobile|    NULL|  20000.0|
+-------------+----------+---------------+-------+--------+---------+
```

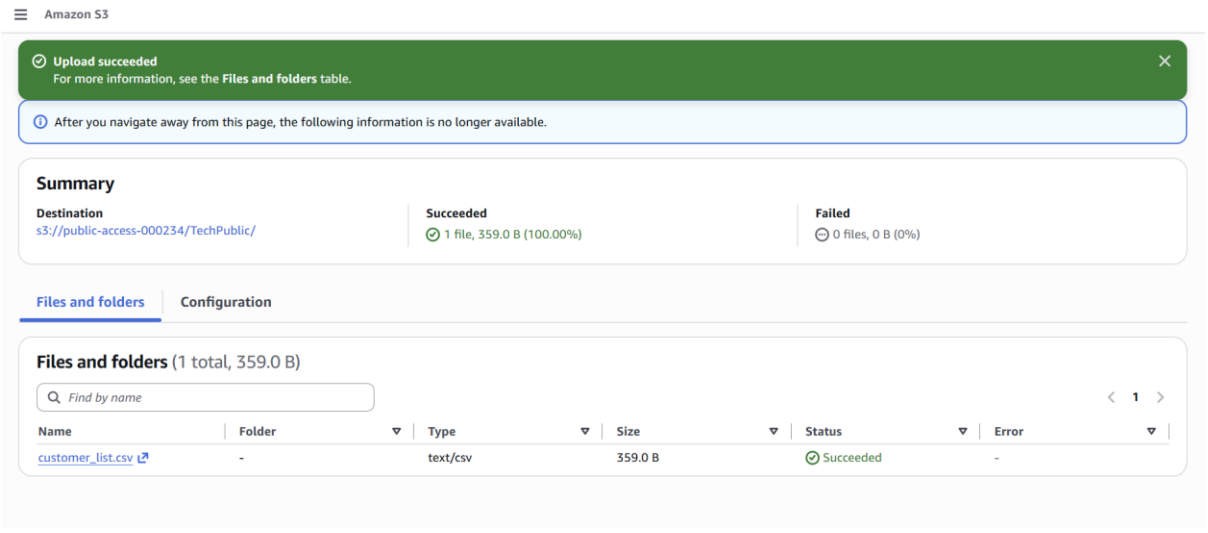
Identify customers who have never placed an order. (1 mark)

```
df_cleaned = df_cleaned.dropna(subset=["TransactionID", "CustomerID", "Product"])
df_cleaned = df_cleaned.fillna({"Quantity": 1, "UnitPrice": 0.0})
df_cleaned.show()
```

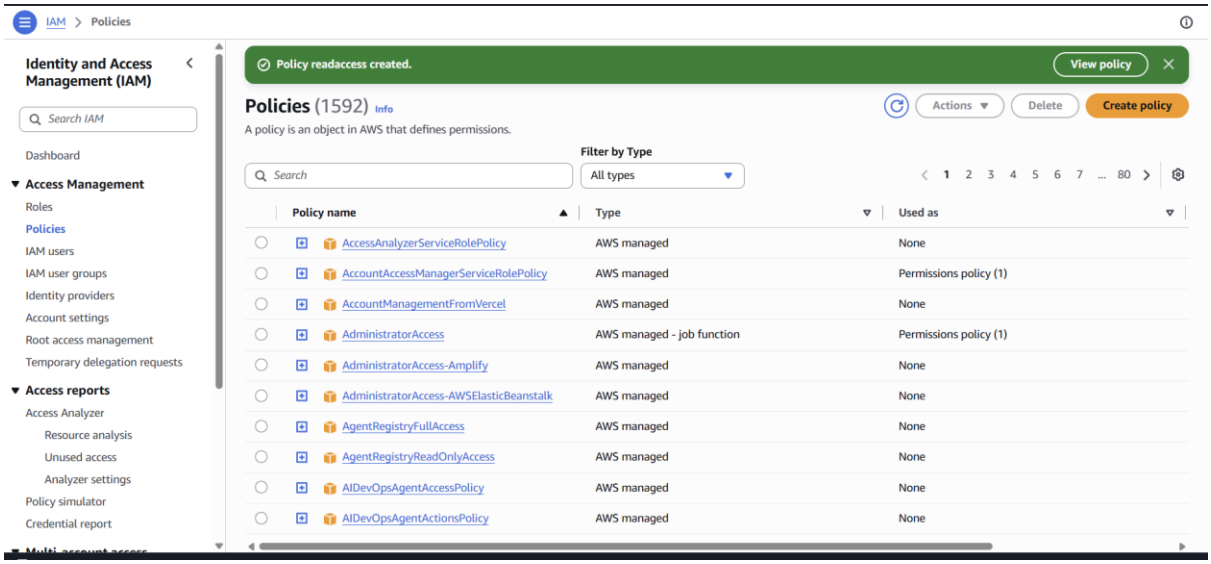
```
+-------------+----------+---------------+-------+--------+---------+
|TransactionID|CustomerID|TransactionDate|Product|Quantity|UnitPrice|
+-------------+----------+---------------+-------+--------+---------+
|            1|     101.0|     2024-01-10| Laptop|       1|  50000.0|
|            2|     102.0|     2024-01-12| Mobile|       1|  20000.0|
|            3|     103.0|     15/01/2024| Tablet|       1|  15000.0|
|            4|     104.0|        invalid| Laptop|       1|  50000.0|
|            7|     107.0|     2024-02-10| Mobile|       1|  20000.0|
+-------------+----------+---------------+-------+--------+---------+
```

SECTION D: AWS CLOUD SERVICES || Maximum Marks: 15 Question 10: S3 and IAM Security (5 Marks)

a. Create an S3 bucket with Block Public Access enabled and upload a sample file. (1 mark)



b. **Design an IAM policy that grants read access to objects in the designated bucket without granting access to unrelated buckets. (2 marks)**



c.Explain how you would verify that an unauthorized user cannot read the file. (1 mark)

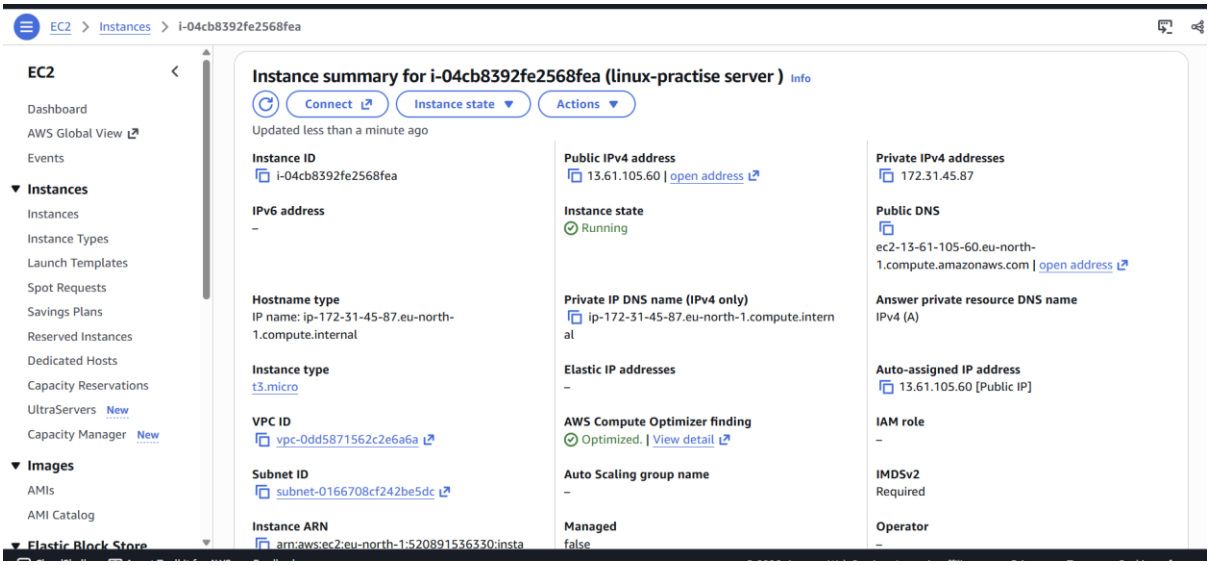
Attempt to access the file using an account that does not have the required permissions. The test passes if access is denied and the user cannot view or open the file.

d. Explain how encryption at rest protects the stored data. (1 mark)

Encryption at rest stores data in an encrypted form on disks or storage devices. Even if an attacker gains physical access to the storage, the data remains unreadable without the correct decryption key.

Question 11: EC2 Troubleshooting and CloudWatch (5 Marks)

a. Check the EC2 instance state and status checks**. (1 mark)**



b. Explain how security groups and route tables can affect application connectivity. (1 mark)

Security groups allow traffic; route tables direct network paths.

c. Explain how to configure a CloudWatch alarm when CPU utilization exceeds 85% for a defined evaluation period.

Create CloudWatch alarm: CPU >85%, set evaluation period, save.

d. Identify one method for investigating the root cause of high CPU utilization. (1 mark)

Check CloudWatch metrics and logs to find CPU spikes.

Question 12: Event-Driven Processing (5 Marks)

a. Identify a suitable service for executing the processing code. (1 mark)

Service: AWS Lambda.

b. Explain how an S3 upload can trigger processing.

Trigger: S3 Event Notification triggers Lambda when a CSV file is uploaded.

Explain the IAM permissions needed to read the input file and write the results. (1 mark)

Permissions: IAM role needs GetObject (read) and PutObject (write) permissions.

Explain how errors and processing logs can be captured. (1 mark)

Logs/Errors: Use Amazon CloudWatch Logs

Explain how to prevent duplicate processing or recursive triggering. (1 mark)

Avoid duplicates: Save results to a different bucket/folder and track processed files.

SECTION E: AWS STEP FUNCTIONS || Maximum Marks: 15
Question 13: State Machine Design (7 Marks)

a. Identify suitable states for validation, processing, storage, and notification. (2 marks)

Suitable States: Use Task states to run AWS Lambda functions or AWS SDK integrations for validation, data transformation/calculation, S3 upload, and SNS notification; optionally add a Choice state after validation for branching and a Fail or Succeed state at termination points

b. Explain how a Choice state can route invalid files to a failure path and valid files to processing. (1 mark)

Choice State Routing: A Choice state inspects a boolean or status field in the validation output (e.g., $.isValid == true) to route valid files to the processing

Task state and invalid files via Default or matching condition directly to a Fail state.

c. Explain how Retry and Catch can handle transient failures and permanent errors. (2 marks)

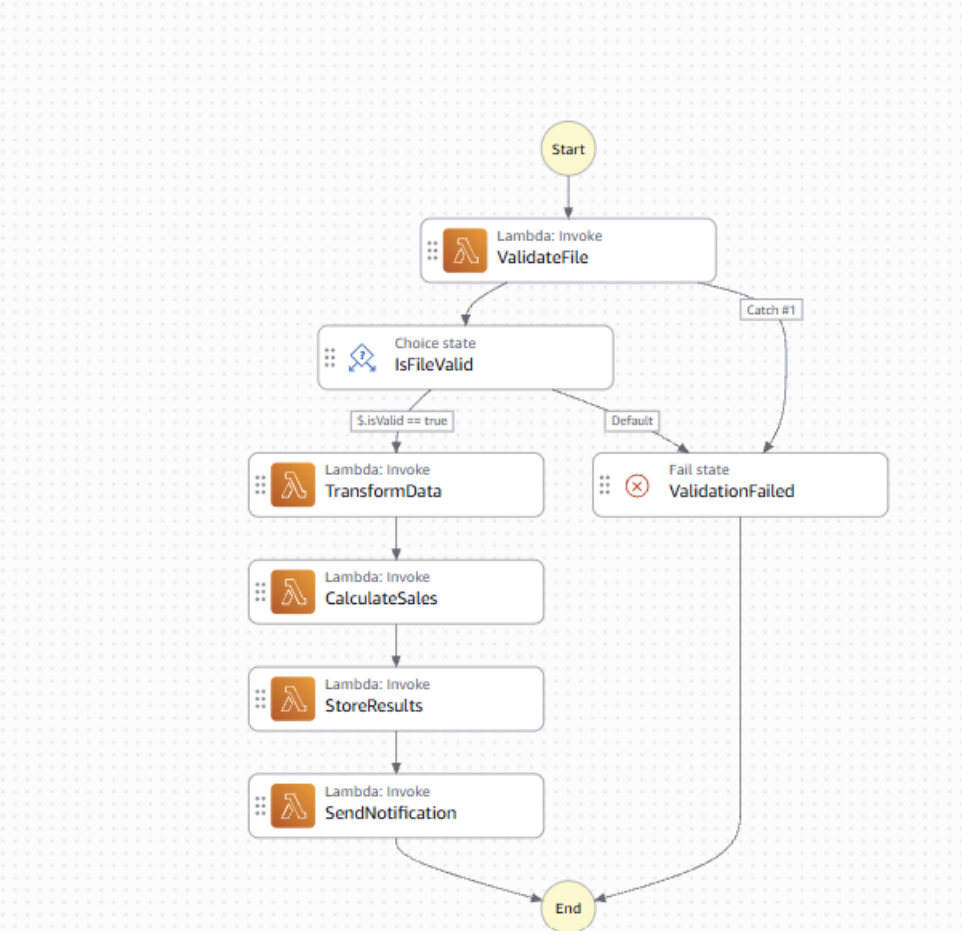
Retry vs. Catch: Retry automatically re-executes a failed state with configurable intervals and backoffs to overcome temporary glitches like network timeouts, while Catch intercepts unrecoverable or persistent errors to cleanly divert execution to a fallback or error-handling state.

d. Explain how the state machine can invoke Lambda functions or other supported AWS services. (1 mark)

Invoking Lambda & AWS Services: The state machine invokes services using a Task state by specifying the target service's ARN in the "Resource" field (such as a Lambda ARN or an optimized service integration like arn:aws:states:::sns:publish).

e. Explain how execution history helps troubleshoot a failed workflow. (1 mark)

Execution History Troubleshooting: The execution history logs a chronological step-by-step audit of state entries, exits, timestamps, exact input/output payloads, and detailed error tracebacks to quickly pinpoint the exact failure point.



Question 14: Step Functions Implementation (8 Marks)

Configure a retry policy for the transformation task with an appropriate error category, retry interval, and maximum attempts. (2 marks)

Validation: A Choice state checks validation results. If the file is invalid, the workflow stops using a Fail state

Configure a Catch transition for the loading task that routes to an error-handling state. (1 mark)

Retry: The TransformData task uses Retry with IntervalSeconds: 2, MaxAttempts: 3, and BackoffRate: 2 to handle temporary failures.

Explain how data from one task is passed to the next task. (1 mark)

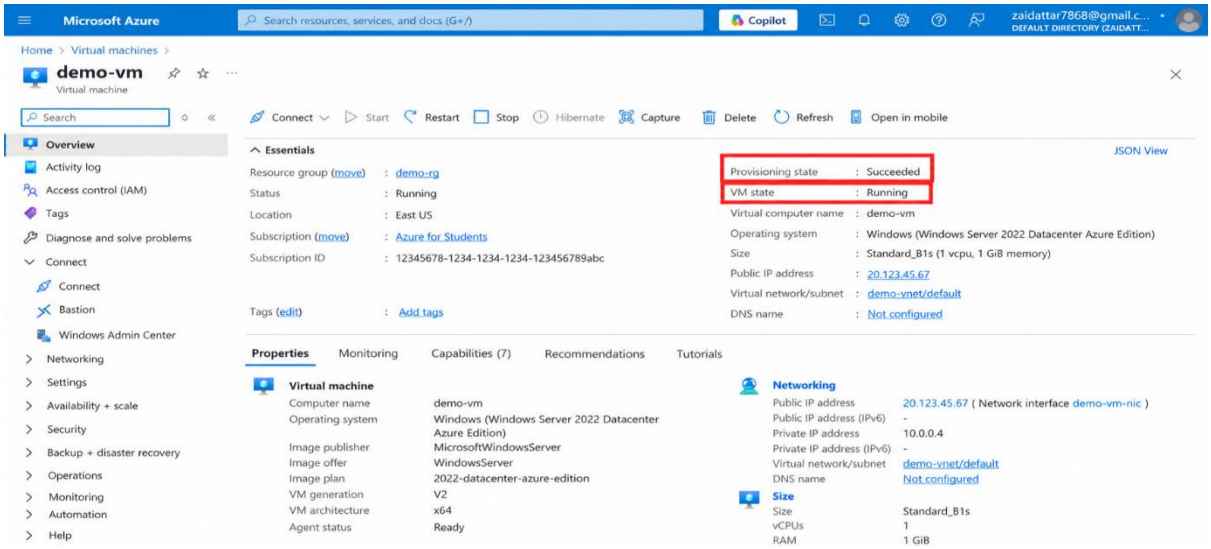
Catch: The LoadResults task uses Catch to route permanent errors to RecordFailure for logging.

Explain how you would test successful and failed executions. (1 mark)

Data passing and testing: Payload. $ passes input between tasks, while Result Path stores task outputs. Execution history helps debug errors and verify successful or failed runs.

Question 16: Azure VM Troubleshooting (5 Marks)

a. Verify the VM's provisioning and power states.



b. How can NSG rules prevent connectivity?

- Network Security Group (NSG) rules control inbound and outbound traffic.
- If inbound rules block ports such as 22 (SSH) or 3389 (RDP), administrators cannot connect to the VM.

c. How do subnet and routing configurations affect connectivity?

- Incorrect subnet configuration or route table entries can block traffic to the VM.
- Verify the subnet, route table, and network connectivity settings.

d. Recommend a secure troubleshooting method.

- Use Azure Bastion to securely connect to the VM through the Azure Portal without exposing its public IP address.
- Alternatively, use Just-in-Time (JIT) VM access to open required ports only when needed, reducing exposure to the public internet.

Question 17: Azure Data Integration

A company receives a CSV file containing daily customer transactions and needs to prepare it for analysis. Design a solution using Azure services.

a. Select an appropriate service to orchestrate data movement.

Check the VM's status in the Azure portal under Virtual Machines. Verify that the provisioning state is Succeeded and the power state is Running.

b. Explain how to copy the CSV file into Azure Data Lake Storage Gen2. (1 mark)

Network Security Group (NSG) rules may block incoming connections if the required port, such as SSH (22) or RDP (3389), is denied. Verify that the inbound rules allow traffic from authorized sources.

c. Describe how to organize files by ingestion date. (1 mark)

Incorrect subnet configurations, route tables, or user-defined routes may prevent traffic from reaching the VM or returning to the client. Check the subnet association, route table, and network connectivity.

d. Explain how to handle malformed records and missing values. (1 mark)

Use Azure Bastion to connect securely to the VM through the Azure portal without assigning a public IP address to the VM. Alternatively, use Azure Serial Console to troubleshoot supported boot and connectivity issues. Restrict NSG rules to authorized sources and avoid opening ports to the entire internet.